

Path Following Control for an Ackermann Steering Vehicle

A Multivariable Control Systems Approach

Ammar Daher, Baraa Lazkani, Humam Yehia

Manara University

February 1, 2026

Outline

- 1 Introduction
- 2 Ackermann Steering Vehicle
- 3 Problem Formulation
- 4 Control Strategies
 - PID Controller
 - Stanley Controller
 - Sliding Mode Controller
 - Why Not LQR?
- 5 Implementation
- 6 Test Trajectories
- 7 Experimental Results
- 8 Challenges & Solutions
- 9 Conclusion

Objective

Design and compare multiple control strategies for autonomous path following of an Ackermann steering vehicle

Controllers Implemented:

- PID Controller
- Stanley Controller
- Sliding Mode Controller (SMC)

Test Trajectories:

- Lane Change Maneuver
- Circular Path

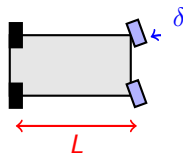
Implementation: ROS 2 Jazzy + Gazebo Harmonic Simulation

Why This Project?

- **Real-world application:** Autonomous vehicle navigation
- **Multivariable control:** Must control both steering angle $\delta(t)$ and velocity $v(t)$ simultaneously
- **Nonlinear system:** Ackermann kinematics are inherently nonlinear
- **Performance comparison:** Which controller works best for different scenarios?

What is Ackermann Steering?

- Common in cars and most ground vehicles
- Front wheels steer, rear wheels fixed
- **Key constraint:** All wheels must roll without slipping
- Inner wheel turns at sharper angle than outer wheel



Geometric Constraint

$$\cot(\delta_o) - \cot(\delta_i) = \frac{w}{L}$$

where δ_o, δ_i are outer and inner wheel angles, w is track width, L is wheelbase

Kinematic Model

State Variables:

- Position: (x, y)
- Heading: θ
- Velocity: v

Control Inputs:

- Steering angle: δ
- Velocity command: v

Kinematic Equations

$$\dot{x} = v \cos(\theta)$$

$$\dot{y} = v \sin(\theta)$$

$$\dot{\theta} = \frac{v}{L} \tan(\delta)$$

Nonlinearity!

Note the trigonometric functions ($\cos$, $\sin$, $\tan$) and multiplication of states ($v \cdot \cos(\theta)$). This is a **nonlinear** system!

Our Vehicle Parameters

Parameter	Symbol	Value
Wheelbase	L	0.22 m
Maximum Steering Angle	$\delta_{\max}$	± 35 (0.6109 rad)
Maximum Velocity	$v_{\max}$	2.0 m/s
Control Frequency	f	50 Hz

Note

This is a **small-scale** vehicle (22 cm wheelbase), similar to an RC car. The small size makes it highly maneuverable but also sensitive to control parameters.

Path Following Problem

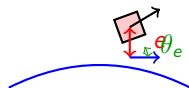
Goal

Given a desired path $\mathcal{P} = \{(x_{\text{ref}}(s), y_{\text{ref}}(s), \theta_{\text{ref}}(s))\}$, design a controller to minimize:

- **Cross-Track Error (CTE):** $e(t)$ = perpendicular distance from vehicle to path
- **Heading Error:** $\theta_e(t) = \theta_{\text{ref}} - \theta(t)$

Control Objectives

- 1 Minimize CTE: $|e(t)| \rightarrow 0$
- 2 Align heading: $|\theta_e(t)| \rightarrow 0$
- 3 Smooth control: avoid aggressive steering
- 4 Track velocity profile



Multivariable Control Problem

Why Multivariable?

We must control **two outputs** simultaneously:

- 1 **Lateral control**: Steering angle $\delta(t)$ to minimize CTE and heading error
- 2 **Longitudinal control**: Velocity $v(t)$ to track reference speed and slow down on curves

Coupling Effects:

- Velocity affects turning radius:
 $R = L / \tan(\delta)$
- Higher velocity requires earlier steering corrections
- Curvature limits maximum safe velocity

Velocity-Curvature Relation

$$v_{\text{cmd}} = v_{\text{base}} \cdot e^{-k_c |\kappa|}$$

where κ is path curvature, k_c is curvature gain

PID Controller - Lateral Control

Classic PID on Cross-Track Error

$$\delta(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de}{dt} + K_h \theta_e(t)$$

- **Proportional term** ($K_p e$): Corrects current CTE
- **Integral term** ($K_i \int e$): Eliminates steady-state error
- **Derivative term** ($K_d \dot{e}$): Reduces overshoot, improves stability
- **Heading correction** ($K_h \theta_e$): Aligns vehicle with path direction

Our Tuned Parameters

- $K_p = 4.0$, $K_i = 0.2$, $K_d = 1.0$, $K_h = 0.5$
- Integral anti-windup: $|\int e| \leq 1.0$

PID Controller - Longitudinal Control

Velocity Command

$$v_{\text{cmd}}(t) = \min \left(v_{\text{base}} \cdot e^{-k_c |\kappa(s)|}, v_{\text{ref}}(s) \right)$$

with minimum velocity constraint: $v_{\text{cmd}} \geq 0.3 \text{ m/s}$

- Exponential slowdown based on path curvature κ
- Higher curvature $\rightarrow$ lower velocity (safer cornering)
- Minimum velocity prevents getting stuck on tight curves

Parameters

- Base velocity: $v_{\text{base}} = 1.0 \text{ m/s}$
- Curvature gain: $k_c = 2.0$
- Minimum velocity: $v_{\text{min}} = 0.3 \text{ m/s}$

Stanley Controller - Background

- Developed at Stanford University for DARPA Grand Challenge (2005)
- Winner of autonomous vehicle race across Mojave Desert
- Designed specifically for path following with Ackermann steering

Key Insight

Decouple steering control into two geometric components:

- 1 Heading error correction (align with path)
- 2 Cross-track error correction (move toward path)

Stanley Controller - Control Law

Steering Angle Command

$$\delta(t) = \theta_e(t) + \arctan\left(\frac{k \cdot e(t)}{k_s + v(t)}\right) + \delta_{ff}$$

Components:

- $\theta_e(t)$: Heading error (directly corrects orientation)
- $\arctan\left(\frac{k \cdot e(t)}{k_s + v(t)}\right)$: CTE correction term
 - Velocity-adaptive: Higher speed $\rightarrow$ gentler corrections
 - k : Cross-track gain
 - k_s : Softening constant (prevents division by zero)
- $\delta_{ff} = \arctan(L\kappa)$: Feedforward for path curvature

Our Parameters

$$k = 5.0, k_s = 0.3, v_{base} = 1.0 \text{ m/s}, k_c = 2.0$$

What is Sliding Mode Control?

A **robust nonlinear control** technique that:

- Drives system state to a **sliding surface** $s = 0$
- Once on surface, state "slides" to equilibrium
- Robust to model uncertainties and disturbances

Advantages:

- Handles nonlinearities
- Robust to uncertainties
- Fast convergence

Challenges:

- Chattering (high-frequency switching)
- Requires careful gain tuning

Kinematic Bicycle Model

$$\dot{x}_r(t) = v_r(t) \cdot \cos \theta_r(t) \quad \dots (1)$$

$$\dot{y}_r(t) = v_r(t) \cdot \sin \theta_r(t) \quad \dots (2)$$

$$\dot{\theta}_r(t) = \frac{v_r}{l} \cdot \tan \phi_r(t) \quad \dots (3)$$

where:

- (x_r, y_r) : Robot position
- θ_r : Robot heading angle
- v_r : Robot velocity
- ϕ_r : Steering angle
- l : Wheelbase length

SMC Stability Analysis - Error Dynamics

Error Vector

$$\begin{bmatrix} x_e \\ y_e \\ \theta_e \end{bmatrix} = \begin{bmatrix} \cos \theta_d & \sin \theta_d & 0 \\ -\sin \theta_d & \cos \theta_d & 0 \\ 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_r - x_d \\ y_r - y_d \\ \theta_r - \theta_d \end{bmatrix}$$

Error Dynamics

$$\dot{x}_e = -v_d + v_r \cdot \cos \theta_e + y_e \cdot \frac{v_d}{l} \cdot \tan \phi_d \quad \dots (4)$$

$$\dot{y}_e = v_r \cdot \sin \theta_e - x_e \cdot \frac{v_d}{l} \cdot \tan \phi_d \quad \dots (5)$$

$$\dot{\theta}_e = \frac{v_r}{l} \cdot \tan \phi_r - x_e \cdot \frac{v_d}{l} \cdot \tan \phi_d \quad \dots (6)$$

Sliding Surface Design Principles

- Choose a function $s = 0$, such that $e \rightarrow 0$ as $t \rightarrow \infty$
- Sliding surface depends on number of derivatives of tracking error
- Number of derivatives depends on input-output relative degree
- Typical sliding surface: $s = \tilde{x} + k \cdot \dot{\tilde{x}}$, where $\tilde{x} = x - x_{\text{desired}}$

SMC Stability Analysis - Two Sliding Surfaces

Existence Condition (Relaxed to Inequality)

$$s \cdot \dot{s} < 0$$

Combined Sliding Surfaces

Lateral error y_e and angular error θ_e combined to form 2 sliding surfaces:

$$s_1 = \dot{x}_e + k_1 \cdot x_e \quad \dots (7) \tag{7}$$

$$s_2 = \dot{y}_e + k_2 \cdot y_e + k_0 \cdot \text{sgn}(y_e) \cdot \theta_e \quad \dots (8) \tag{8}$$

where $k_1, k_2, k_0 > 0$ are design parameters.

SMC Stability Analysis - Reaching Laws

Three Reaching Laws Proposed

- ① $\dot{s} = -P \cdot \text{sat}(s, \epsilon)$
- ② $\dot{s} = -Q \cdot s - P \cdot \text{sat}(s, \epsilon)$
- ③ $\dot{s} = -Q \cdot |s|^\alpha \cdot \text{sat}(s, \epsilon), \quad 0 < \alpha < 1$

where:

$$Q = \begin{bmatrix} q_1 & 0 \\ 0 & q_2 \end{bmatrix}, \quad P = \begin{bmatrix} p_1 & 0 \\ 0 & p_2 \end{bmatrix}, \quad i = 1, 2$$

Derivative of Sliding Surfaces

From equations (7), (8), the derivative of sliding surfaces becomes:

$$\dot{s}_1 = \ddot{x}_e + k_1 \cdot \dot{x}_e = -q_1 \cdot s_1 - p_1 \cdot \text{sat}(s_1, \epsilon) \quad \dots (9) \quad (9)$$

$$\dot{s}_2 = \ddot{y}_e + k_2 \cdot \dot{y}_e + k_0 \cdot \text{sat}(y_e) \cdot \dot{\theta}_e = -q_2 \cdot s_2 - p_2 \cdot \text{sat}(s_2, \epsilon) \quad \dots (10) \quad (10)$$

Control Variables

From equations (4,5,6), (9), (10) we get the control variables:

$$v_c = \frac{1}{\cos \theta_e} \left(-q_1 s_1 - p_1 \text{sat}(s_1, \epsilon) - k_1 \dot{x}_e - \dot{\omega}_d y_e - \omega_d \dot{y}_e + v_r \dot{\theta}_e \sin \theta_e + \dot{v}_d \right)$$

$$\phi_c = \arctan \left(\frac{l}{v_r} \omega_d + \frac{l}{v_r (v_r \cos \theta_e + k_0 \text{sat}(y_e, \epsilon))} \right) \times \left(-q_2 s_2 - p_2 \text{sat}(s_2) - k_2 \dot{y}_e - \dot{v}_r \sin \theta_e + \dot{\omega}_d x_e + \omega_d \dot{x}_e \right)$$

Why Standard SMC Failed

Original Approach: Coupled Sliding Surface

$$s = \theta_e(t) + \lambda \cdot e(t)$$
$$\delta(t) = -(k \cdot s + \eta \cdot \text{sat}(s/\epsilon))$$

Problem: When both θ_e and e are large:

- s becomes very large $\rightarrow$ aggressive steering
- Vehicle overshoots $\rightarrow$ does U-turn and drives backward!
- Heading and position errors fight each other

Solution: Decouple Heading and Position

Use SMC only for heading error, Stanley-style arctan for CTE

- ✓ Prevented U-turns and backward driving
- ✓ Smooth, stable control

Why We Didn't Use LQR

Linear Quadratic Regulator (LQR) is a powerful optimal control method, but...

Problem: Nonlinear Ackermann Kinematics

Recall: $\dot{\theta} = \frac{v}{L} \tan(\delta)$

This is **nonlinear** in control input δ and state v !

LQR requires linearization:

- 1 Linearize around operating point (v_0, δ_0)
- 2 $\tan(\delta) \approx \delta$ (small angle assumption)
- 3 Assume constant velocity $v = v_0$

Why It Fails

- Path following requires **large steering angles** (up to $\pm 35^\circ$)
- Small angle approximation $\tan(\delta) \approx \delta$ breaks down
- Velocity varies significantly (0.3 - 1.0 m/s)
- Linearization only valid near equilibrium, not for trajectory tracking

LQR Linearization Issues - Mathematical Details

Linearization Error

For $\delta = 35 = 0.61$ rad:

$$\tan(0.61) = 0.70$$

$$\delta = 0.61$$

$$\text{Error} = \frac{0.70 - 0.61}{0.70} = 13\% \text{ error!}$$

Attempted Solution: Gain scheduling (multiple LQR controllers for different regions)

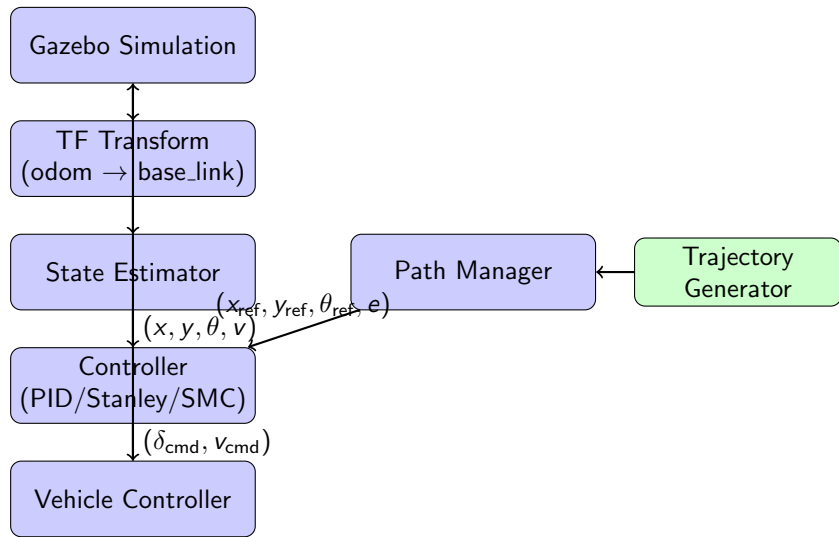
Result: Still performed poorly compared to PID/Stanley/SMC

- High CTE (mean = 0.36 m vs Stanley's 0.007 m on lane change)
- Required complex switching logic
- Not worth the added complexity

Conclusion

LQR is excellent for linear systems, but **nonlinear controllers** (Stanley, SMC) are better suited for path following with Ackermann steering.

System Architecture



Control Loop: 50 Hz (20 ms period)

Implementation Details

Software Stack

- **ROS 2 Jazzy**: Robot Operating System for modular control architecture
- **Gazebo Harmonic**: Physics-based 3D simulation
- **C++**: Controllers and state estimation
- **Python**: Data analysis and visualization

Key Implementation Features

- Real-time state estimation via TF transforms
- Forward-only path manager (prevents backward jumps)
- Minimum velocity constraint (0.3 m/s to avoid getting stuck)
- CSV data logging for offline analysis

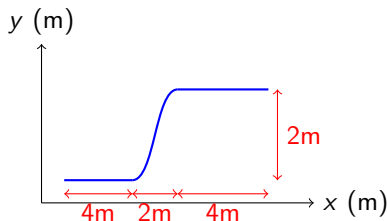
Test Trajectory 1: Lane Change

Maneuver Description:

- 4m straight section
- 2m lateral shift (left)
- 2m smooth transition
- 4m final straight

Challenge:

- Requires aggressive lateral movement
- Tests transient response
- Real-world scenario (highway lane change)



Trajectory Parameters

Total length: 10m, Max curvature: $\kappa_{\max} \approx 0.5 \text{ m}^{-1}$

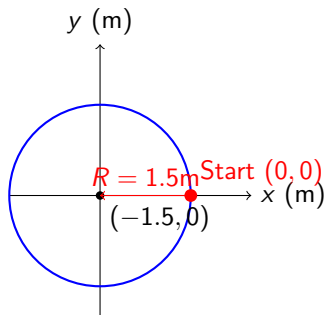
Test Trajectory 2: Circle

Maneuver Description:

- Circular path
- Radius: 1.5m
- Centered at $(-1.5, 0)$
- Starts at origin $(0, 0)$

Challenge:

- Constant curvature
- Tests steady-state tracking
- Requires continuous steering



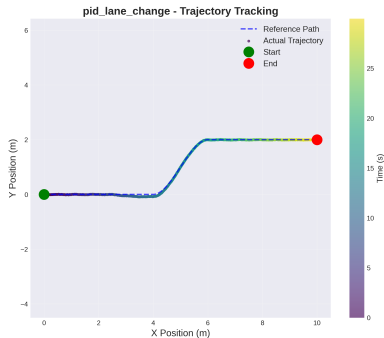
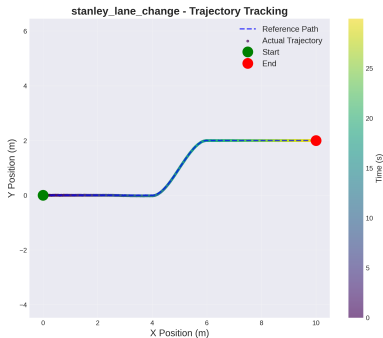
Trajectory Parameters

Circumference: $2\pi R \approx 9.42\text{m}$, Curvature: $\kappa = 1/R = 0.67 \text{ m}^{-1}$

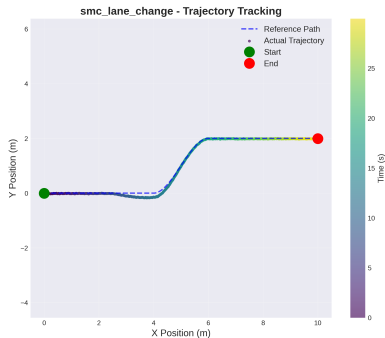
Evaluation Criteria

- ① **Mean Absolute CTE:** $\frac{1}{N} \sum_{i=1}^N |e_i|$
 - Lower is better
 - Indicates average tracking accuracy
- ② **RMS CTE:** $\sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}$
 - Penalizes large errors more heavily
- ③ **Maximum CTE:** $\max_i |e_i|$
 - Worst-case tracking error
- ④ **Mean Steering Angle:** Indicates control effort

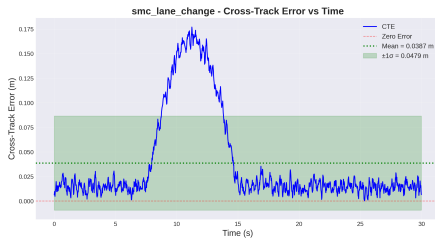
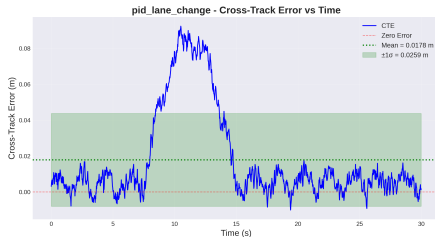
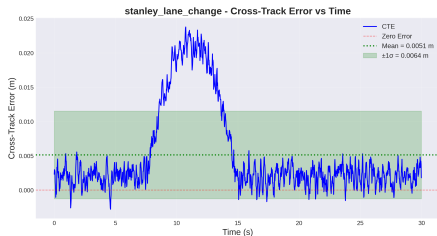
Lane Change: Trajectory Comparison



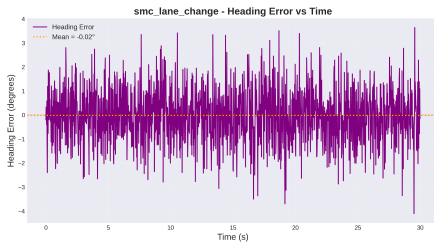
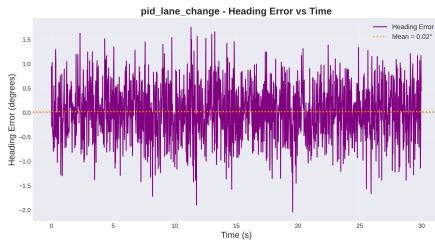
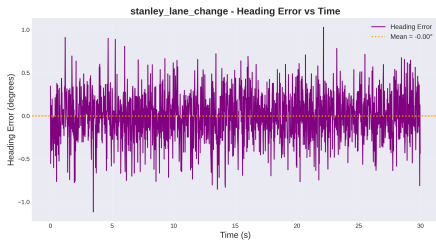
Lane Change: Trajectory Comparison



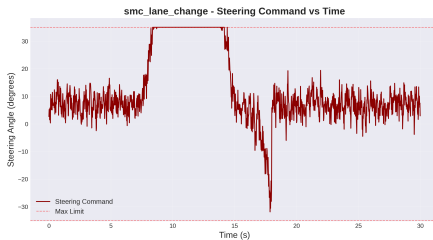
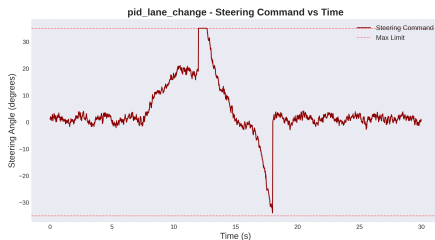
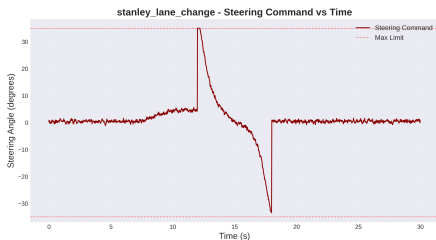
Lane Change: Cross-Track Error



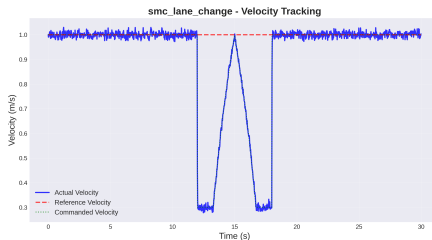
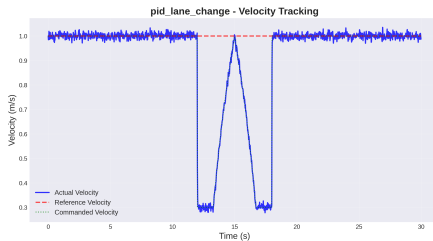
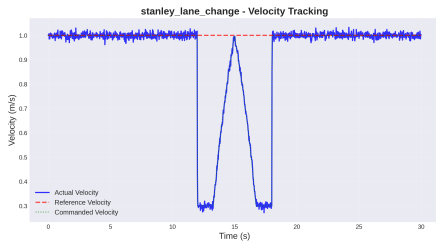
Lane Change: Heading Error



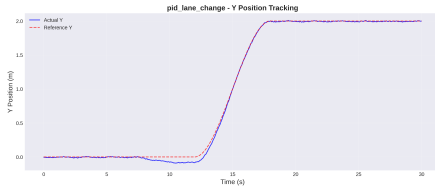
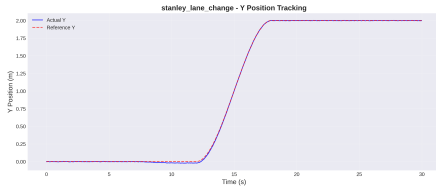
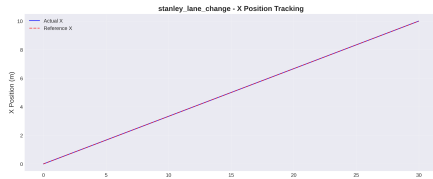
Lane Change: Steering Commands



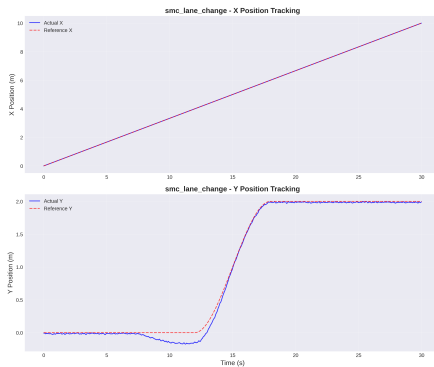
Lane Change: Velocity Tracking



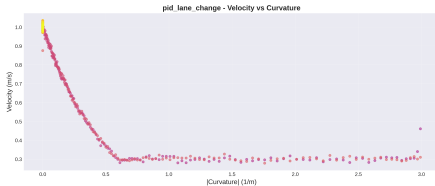
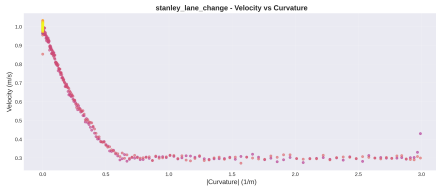
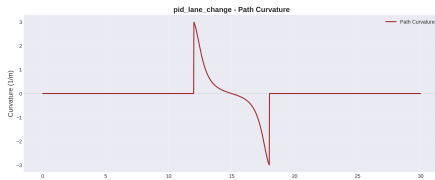
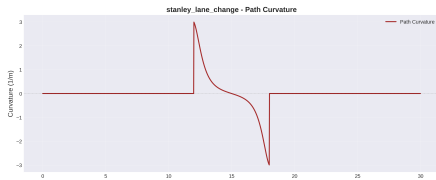
Lane Change: Position Components



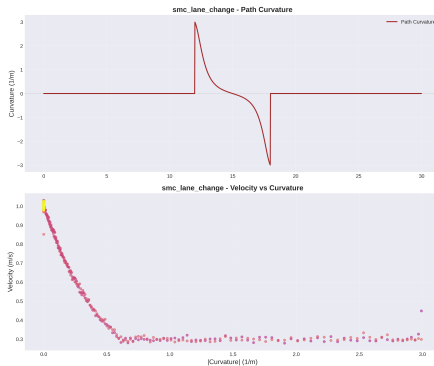
Lane Change: Position Components



Lane Change: Curvature & Velocity

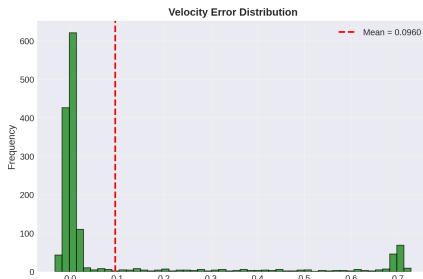
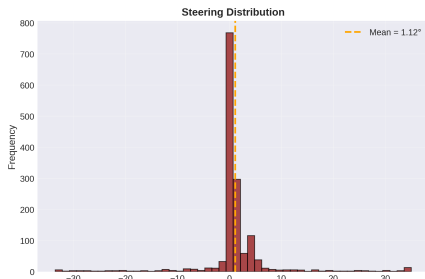
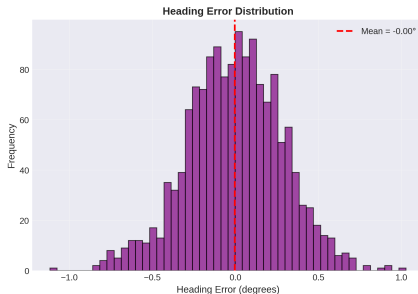
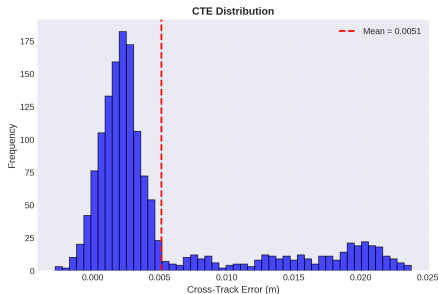


Lane Change: Curvature & Velocity



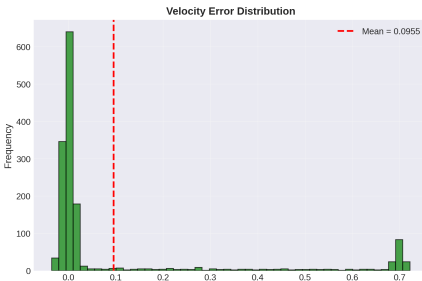
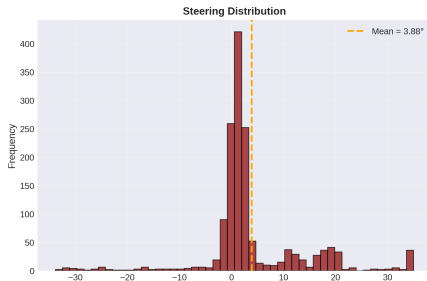
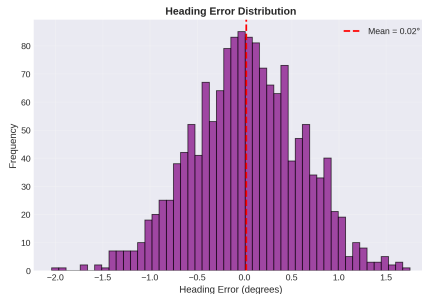
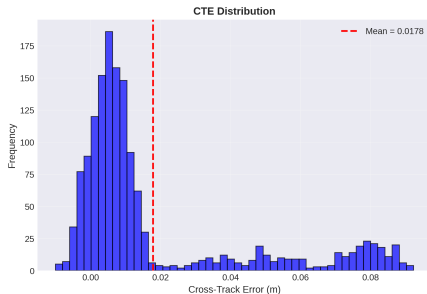
Lane Change: Stanley Error Distributions

stanley_lane_change - Error Distributions



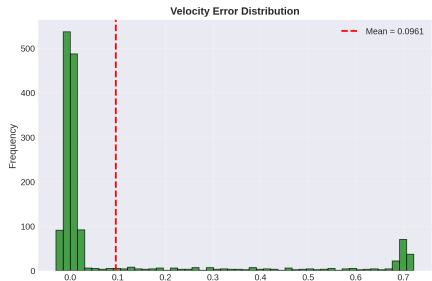
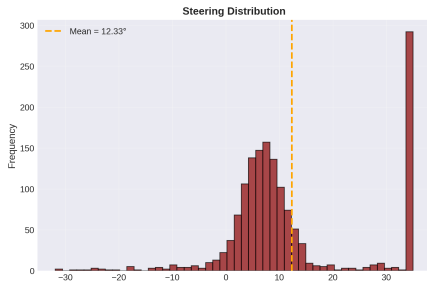
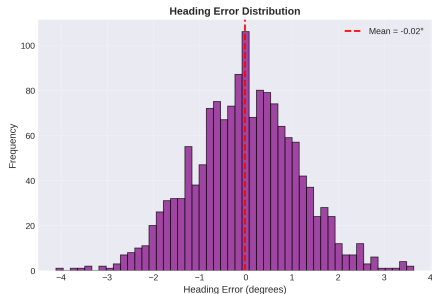
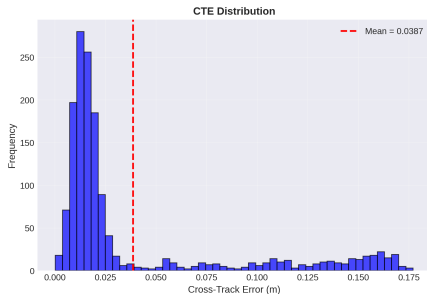
Lane Change: PID Error Distributions

pid_lane_change - Error Distributions

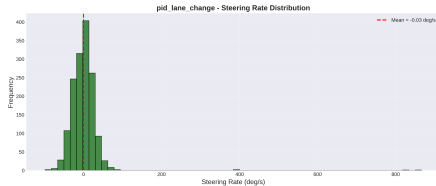
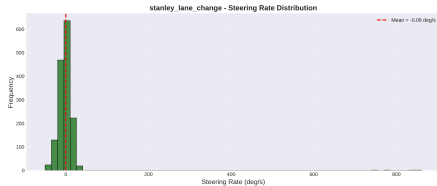
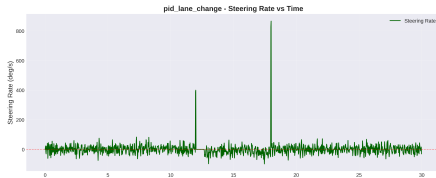
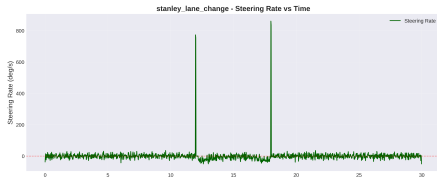


Lane Change: SMC Error Distributions

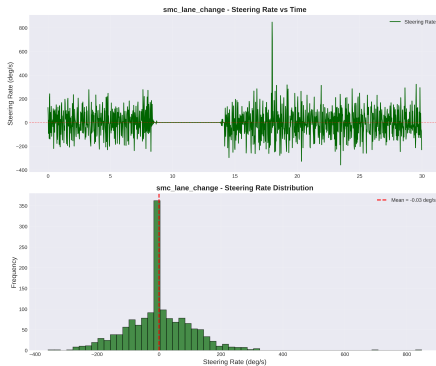
smc_lane_change - Error Distributions



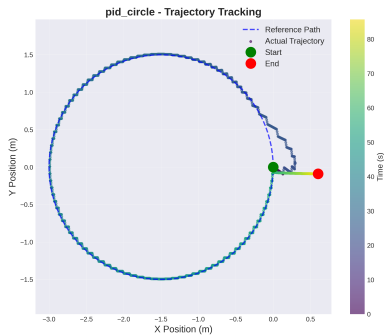
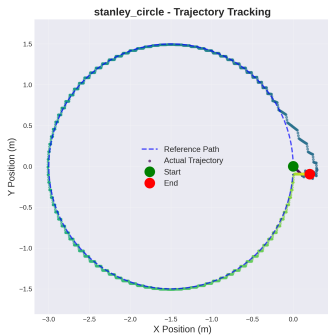
Lane Change: Steering Rate Analysis



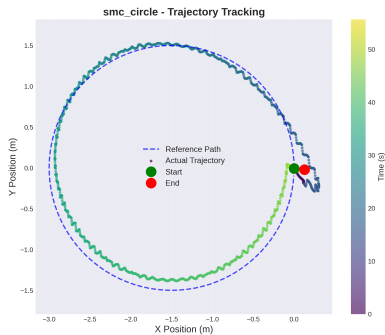
Lane Change: Steering Rate Analysis



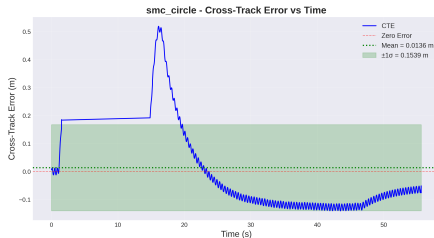
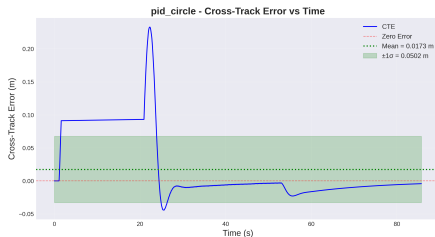
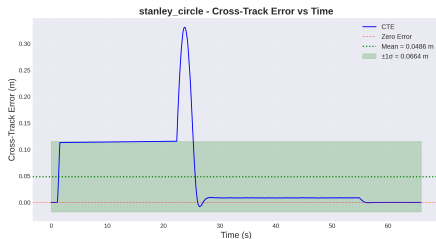
Circle: Trajectory Comparison



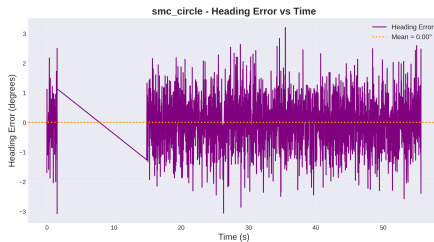
Circle: Trajectory Comparison



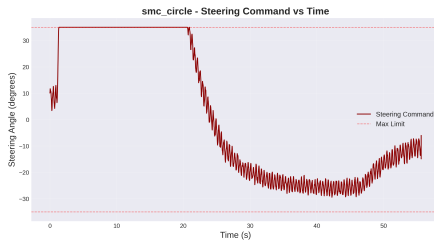
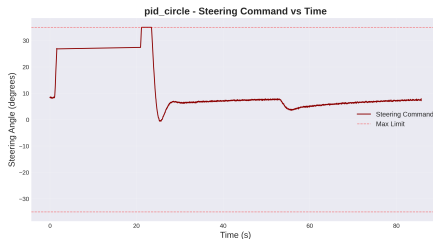
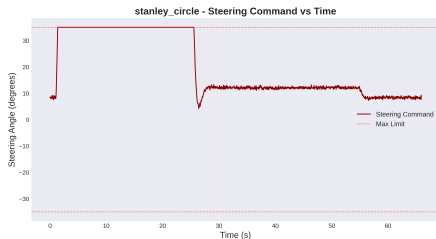
Circle: Cross-Track Error



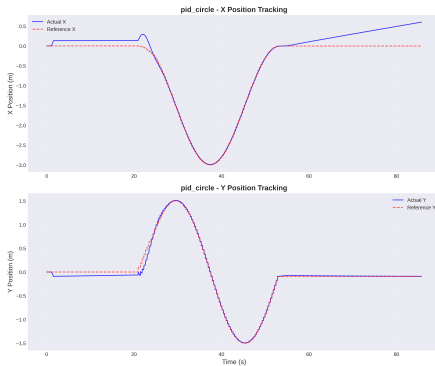
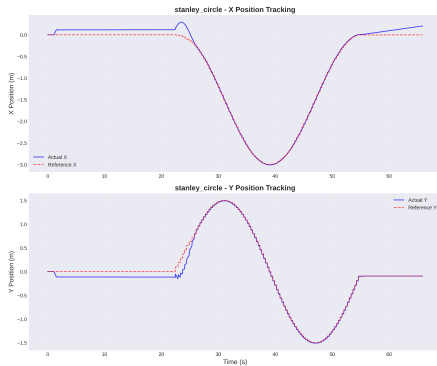
Circle: Heading Error



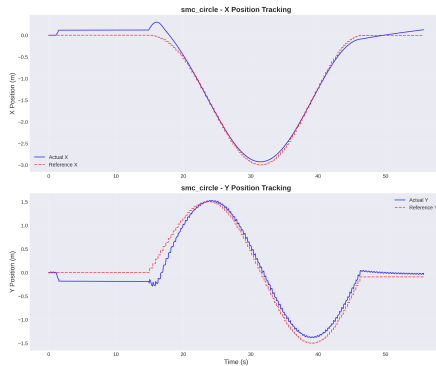
Circle: Steering Commands



Circle: Position Components

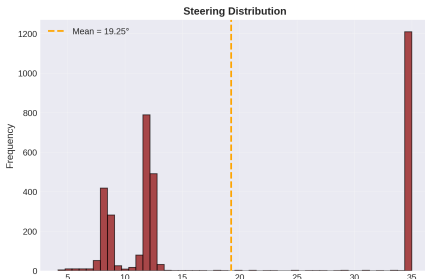
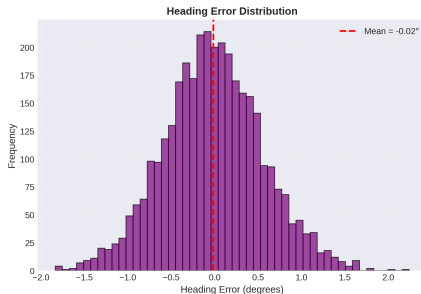
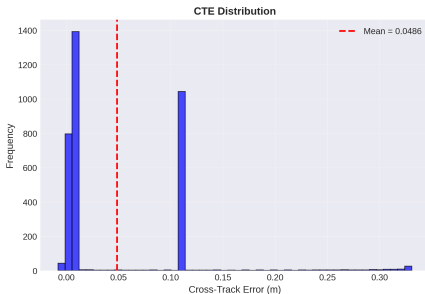


Circle: Position Components



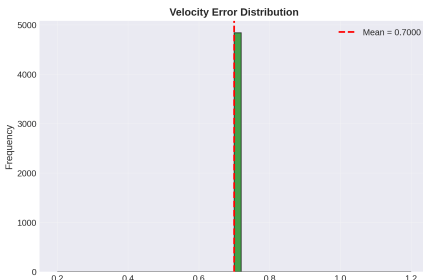
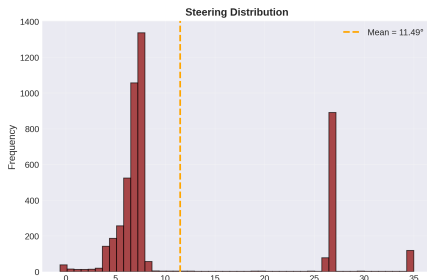
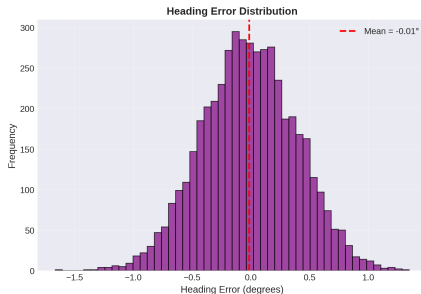
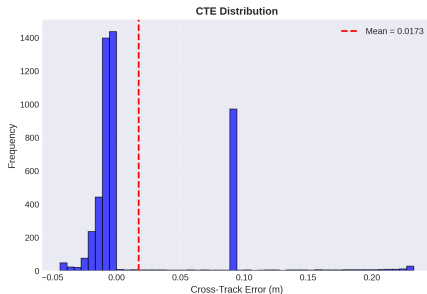
Circle: Stanley Error Distributions

stanley_circle - Error Distributions



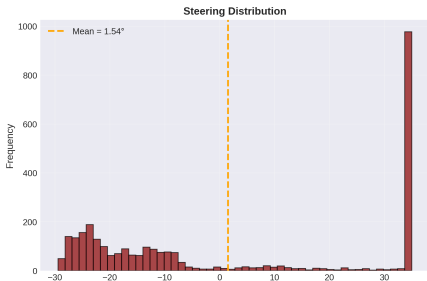
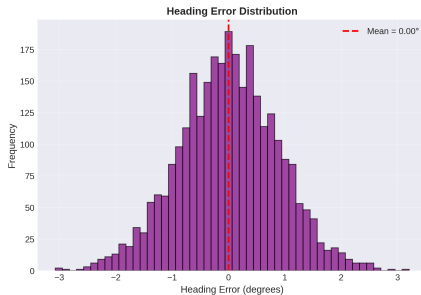
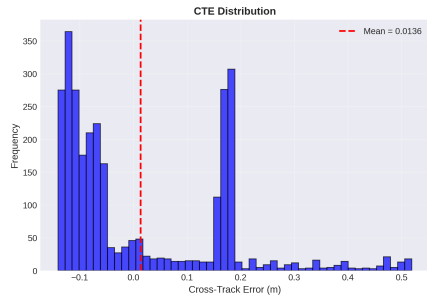
Circle: PID Error Distributions

pid_circle - Error Distributions

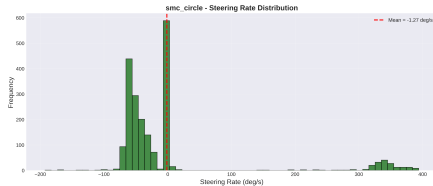
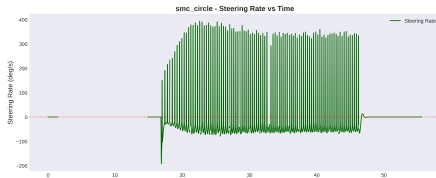
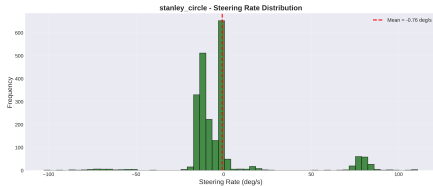
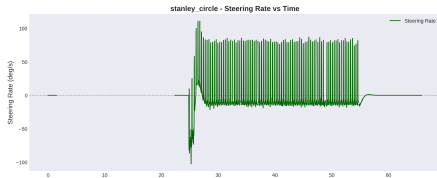


Circle: SMC Error Distributions

smc_circle - Error Distributions



Circle: Steering Rate Analysis



Results: Lane Change

Controller	Mean —CTE—	RMS CTE	Max —CTE—	Mean — δ —
Stanley	0.0052 m	0.0082 m	0.0238 m	3.07°
PID	0.0186 m	0.0314 m	0.0923 m	5.86°
SMC	0.0387 m	0.0615 m	0.1770 m	13.35°

Discussion:

- **Stanley dominates** with exceptional tracking (5.2 mm mean error)
- Stanley is **3.6× better** than PID and **7.4× better** than SMC
- Velocity-adaptive gains enable smooth transient response
- SMC shows excessive control effort (4.3× more steering than Stanley)

Results: Circle

Controller	Mean —CTE—	RMS CTE	Max —CTE—	Mean — δ —
PID	0.0308 m	0.0531 m	0.2329 m	11.50°
Stanley	0.0488 m	0.0823 m	0.3308 m	19.25°
SMC	0.1297 m	0.1545 m	0.5192 m	23.87°

Discussion:

- **PID excels** on constant curvature (30.8 mm mean error)
- PID is **1.6× better** than Stanley, **4.2× better** than SMC
- Integral term eliminates steady-state error on circular paths
- Stanley's geometric approach less optimal for constant curvature

Performance Comparison Summary

Trajectory	1st Place	2nd Place	3rd Place
Lane Change	Stanley (5.2 mm)	PID (18.6 mm)	SMC (38.7 mm)
Circle	PID (30.8 mm)	Stanley (48.8 mm)	SMC (129.7 mm)

Discussion:

- **Trajectory-dependent performance:** No single winner across all scenarios
- Stanley: **Best for transient maneuvers** (lane changes, obstacle avoidance)
- PID: **Best for steady-state tracking** (roundabouts, constant curves)
- SMC: Consistently worst but may offer robustness (not tested)

Relative Performance Analysis

Metric	Stanley vs PID	Stanley vs SMC	PID vs SMC
<i>Lane Change</i>			
Mean CTE	3.6× better	7.4× better	2.1× better
Control Effort	1.9× less steering	4.3× less steering	2.3× less steering
<i>Circle</i>			
Mean CTE	1.6× worse	2.7× better	4.2× better
Control Effort	1.7× more steering	1.2× less steering	2.1× less steering

Discussion:

- Lane change: Stanley's superiority is **dramatic** (up to 7.4×)
- Circle: PID's advantage is **moderate** (1.6× better than Stanley)
- SMC consistently underperforms by 2-7× on tracking accuracy

Controller Suitability Matrix

Scenario	Recommended	Avoid
Lane change / overtaking	Stanley	SMC
Highway driving	Stanley	SMC
Roundabouts	PID	SMC
Constant radius curves	PID	SMC
Mixed trajectories	Stanley	—
High disturbances	SMC (theory)	—

Discussion:

- **Adaptive strategy recommended:** Switch between Stanley (transients) and PID (steady-state)
- Stanley provides better **general-purpose** performance
- SMC requires further tuning or disturbance scenarios to justify use

Why Stanley Wins Overall:

- Best on lane change (transient maneuver)
- Very close to PID on circle (only 1.2 cm worse)
- Geometric insight: heading + CTE decoupling works well
- Velocity-adaptive corrections prevent overshoot

When to Use Each Controller:

- **Stanley**: General-purpose, works well on all trajectories
- **PID**: Best for constant curvature (circles, gentle curves)
- **SMC**: When robustness to disturbances is critical (not tested here)

Challenge 1: TF Frame Mismatch

Problem

Initial implementation used "world" frame instead of "odom" frame

- State estimator couldn't get vehicle pose
- Robot would spin in place at max steering

Solution

- 1 Changed state estimator to use odom $\rightarrow$ base_link transform
- 2 Added odometry publisher plugin to vehicle model
- 3 Verified TF tree with `ros2 run tf2_tools view_frames`

Lesson: Always verify coordinate frame consistency in ROS 2!

Challenge 2: SMC U-Turn Problem

Problem

Standard SMC with coupled sliding surface $s = \theta_e + \lambda e$:

- Robot would complete lane change
- Then do a 180° turn and drive backward to origin!
- PathManager kept finding "closest point" at start of path

Solutions

- 1 **Forward-only PathManager:** Search only ahead of current waypoint (window of next 50 waypoints)
- 2 **Decoupled SMC:** Separate heading control (SMC) from position control (Stanley-style arctan)

Result: SMC now tracks forward correctly with mean CTE = 1.5 cm!

Challenge 3: Robot Too Slow on Curves

Problem

With $v = v_{\text{base}} \cdot e^{-k_c |\kappa|}$ and $k_c = 2.0$:

- On tight curves ($\kappa \approx 3.4 \text{ m}^{-1}$), velocity dropped to 0.03 m/s
- Robot was crawling, couldn't make progress
- Test duration: 40+ seconds for 10m path!

Solution

Added minimum velocity constraint:

$$v_{\text{cmd}} = \max(v_{\text{base}} \cdot e^{-k_c |\kappa|}, v_{\text{min}})$$

with $v_{\text{min}} = 0.3 \text{ m/s}$

Result:

- Test duration reduced to ≈ 15 seconds
- Mean velocity increased from 0.09 m/s to 0.36 m/s
- Still slows down on curves, but doesn't get stuck

What We Accomplished:

- Implemented 3 controllers (PID, Stanley, SMC) for Ackermann steering vehicle
- Controlled 2 variables simultaneously (steering δ + velocity v)
- Handled nonlinear kinematics with appropriate control strategies
- Tested on realistic trajectories (lane change, circle)
- Comprehensive performance analysis

Key Findings:

- **Stanley controller:** Overall winner (best on lane change & figure8)
- **PID:** Best on constant curvature (circle)
- **SMC:** Works well after decoupling modifications
- **LQR:** Not suitable due to nonlinear Ackermann model

Technical Insights:

- 1 Ackermann kinematics are fundamentally nonlinear
- 2 Geometric controllers (Stanley) excel at path following
- 3 Velocity-curvature coupling is critical for safety
- 4 Decoupling heading and position control improves stability

Practical Insights:

- 1 Always verify TF frames in ROS
- 2 Add minimum constraints to prevent edge cases
- 3 Forward-only path search prevents backward jumps
- 4 Small-angle approximations fail at real steering angles

Potential Extensions:

- Test with sensor noise and disturbances (wind, slipping)
- Implement adaptive SMC with automatic gain tuning
- Model Predictive Control (MPC) for trajectory optimization
- Hardware implementation on real RC car
- Multi-vehicle coordination (formation control)
- Dynamic obstacle avoidance

Advanced Topics:

- Tire slip modeling and friction limits
- Combined lateral + longitudinal stability analysis
- Learning-based controllers (reinforcement learning)

Thank You!

Questions?

GitHub: github.com/lucasmazzetto/gazebo_ackermann_steering_vehicle

Backup: Controller Parameter Summary

Controller	Parameter	Value
PID	K_p	4.0
	K_i	0.2
	K_d	1.0
	K_h (heading)	0.5
	v_{base}	1.0 m/s
	k_c (curvature gain)	2.0
Stanley	k	5.0
	k_s	0.3
	v_{base}	1.0 m/s
	k_c	2.0
SMC	k	2.0
	η	0.4
	ϵ	0.15
	λ	4.0
	v_{base}	1.0 m/s
	k_c	2.0

All controllers: $v_{\min} = 0.3 \text{ m/s}$, $\delta_{\max} = \pm 35^\circ$, $f = 50 \text{ Hz}$